

Project Planning and Objectives

Contextualization

Large Language Model (LLM) training workloads can span several weeks and involve models with billions of parameters. Upon failure of one or more nodes, any unsaved state is lost and training must restart from scratch. To mitigate this, the model state (weights, activations, optimizer state) is periodically checkpointed to a shared Parallel File System (PFS) such as Lustre.

However, checkpointing directly to Lustre creates two compounding problems:

- Writing large checkpoint files from multiple nodes simultaneously causes significant I/O contention at the PFS, degrading throughput for all concurrent jobs — the noisy neighbour problem.
- Checkpoint write latency is dominated by Lustre throughput, which stalls training for the duration of the write.

Motivation

- **Resilience**

As HPC clusters grow in size, the **Mean Time Between Failures (MTBF)** decreases. At large scales, hardware or software failures can occur frequently, making reliable checkpointing essential to ensure training progress is not lost.

- **Checkpoint Bottleneck**

With the increase of the number of parameters, the checkpoint size increases drastically, and the storage itself is a problem, even in an HPC context, increasing cost and time.

- **PFS Contention**

Writing those massive files in a shared and parallel file system creates a contention effect where the nodes compete for I/O bandwidth, leading to significant stalls (training time).

- **Storage Overhead**

Maintaining multiple checkpoints for fault tolerance and recovery requires significant storage capacity. This increases both infrastructure costs and the complexity of managing checkpoint data over long training runs.

Objectives

We expect to produce a two-tier, policy-driven checkpoint staging system built with software-defined storage practices. The key idea is to transparently redirect checkpoint writes from PyTorch to node-local SSDs, then orchestrate asynchronous flushing to Lustre under the control of a centralized policy engine.

This approach is inspired by the Software-Defined Storage (SDS) paradigm: I/O mechanisms are separated from the policies that govern them. The control plane has global visibility across all training nodes and can enforce rate limits, scheduling policies without modifying the training code.

Proposed Solution

The system is composed of two main components — a **centralized orchestrator** and a **client per training node** — plus the persistent **Lustre file system** as the final storage destination.

Orchestrator

The orchestrator is a **single centralized process** responsible for coordinating all checkpoint activity across the training job. Its responsibilities are:

- Maintaining a global view of checkpoint state across all training nodes.
- Receiving notifications from each client (checkpoint written to SSD, flush complete).
- Enforcing **pluggable flush policies** that control when and at what rate each client is allowed to flush its staged checkpoint to Lustre.
- Solving the **noisy neighbour problem** by globally coordinating Lustre write bandwidth across concurrent jobs — based on PADLL's QoS control approach.

Client (per training node)

Each training node runs a **client process** co-located with the PyTorch training process:

- Intercepts POSIX I/O calls (`open` , `write` , `close`) **transparently**, without modifying PyTorch source code.
- Classifies requests as checkpoint writes (by file path pattern, e.g. `/ckpt/**`) versus other I/O traffic.
- Redirects classified checkpoint writes to the **node-local SSD** (`/nvme/local/ckpts/`).
- Communicates with the orchestrator: reports checkpoint-ready events and receives flush authorization.
- Performs **background asynchronous flush** of staged checkpoints to Lustre when instructed by the orchestrator.

Local SSD Staging Tier

Each node's local NVMe SSD acts as a **fast intermediate buffer**. Checkpoint writes land here first — with low latency and no contention — then drain to Lustre asynchronously without blocking training.

PyTorch Integration

A minimal wrapper is placed around the checkpointing call in the training script. It sets the checkpoint output path to the local directory monitored by the PAIO stage. This is the only change required to the training code — all interception and orchestration should be transparent below this point.

Activity Plan

#	Task	Owner	Status
Phase 1: Foundation & Setup (Weeks 1–2)			
1	Study PAIO and PADLL source code and papers	Team	Planned
2	Set up HPC cluster environment and Lustre access	Team	Planned
3	Run existing PAIO benchmarks (baseline I/O performance)	Team	Planned
4	Define checkpoint classification rules (path patterns)	Team	Planned

#	Task	Owner	Status
Phase 2: Client Integration (Weeks 3–4)			
1	Implement checkpoint-aware PAIO client stage	Team	Planned
2	Redirect PyTorch <code>torch.save()</code> writes to local SSD via PAIO	Team	Planned
3	Validate correctness: reload checkpoint from local SSD	Team	Planned
4	Measure write latency: direct Lustre vs. local SSD (single node)	Team	Planned

#	Task	Owner	Status
Phase 3: Orchestrator & Policies (Weeks 5–7)			
1	Design orchestrator-client communication protocol (gRPC/socket)	Team	Planned
2	Implement orchestrator (checkpoint state tracking per node)	Team	Planned
3	Implement pluggable policy interface	Team	Planned
4	Implement rate-limiting policy to reduce Lustre pressure	Team	Planned
5	Test with multiple concurrent training jobs (noisy neighbour)	Team	Planned

#	Task	Owner	Status
Phase 4: Evaluation & Write-up (Weeks 8–10)			
1	Benchmark checkpoint latency across all configurations	Team	Planned
2	Measure PFS I/O pressure under concurrent workloads	Team	Planned
3	Compare against DeepSpeed and SCR baselines	Team	Planned
4	Write final report and prepare presentation	Team	Planned

Benchmark Policy

Environment

- Deucalion HPC Cluster (??? partition)

Tools

- [PyTorch Profiler](#)
- [Flexible I/O tester](#) (Maybe)
- Still researching...

Metrics

- Checkpointing time, size and frequency
- Training time, loss and accuracy
- IO congestion
- CPU/GPU usage
- Memory usage
- ...

Related Work

A. *CheckFreq* - Adaptive Frequency and Pipelined Checkpointing

- **How?**

It uses an algorithmic approach to identify optimal checkpointing boundaries. It pipelines the checkpointing process with training iterations, dynamically adjusting frequency.

- **Problems?**

It optimizes when to save, it still relies on the PFS for the final write, which concern us because of the motivations.

B. *Check-N-Run*

- **How?**

Focuses on “hiding” the cost of I/O by using local node resources (DRAM/SSD) as a temporary staging area, allowing the training to resume immediately while the data is moved.

- **Problems?**

If a node fails before its local data is flushed to the PFS, that checkpoint is lost.

C. *DeepFreeze* (Similar to Monarch)

- **How?**

Uses a transparent storage tiering system. It uses high-speed local tiers (NVMe/PMEM) to accelerate the write path, abstracting the complexity from the user.

- **Problems?**

...

D. *BitSnap*

- **How?**

It focuses on what is being saved rather than where. It introduces Sparsification and Quantization to the checkpointing process. It identifies which weights are “critical” and compresses the rest (e.g., from 16-bit to 4-bit or less) before the data even reaches the I/O layer.

- **Problems?**

While it significantly reduces the size of the files, it introduces a trade-off between compression time (CPU overhead) and accuracy. If the quantization is too aggressive, the model might not converge correctly after a resume. Also, it does not manage the storage hierarchy (PFS congestion) by itself.

E. *FastPersist*

- **How?**

It exploits the tensor immutability during the Forward and Backward passes. While the GPU is busy computing gradients, *FastPersist* proactively moves the “stale” model state from GPU memory to the Host RAM and Local SSD in the background. It also uses a distributed “sharding” approach to parallelize writes across nodes.

- **Problems?**

It requires a carefully managed memory buffer on the Host (RAM) to avoid interfering with the training process. While it hides I/O latency, it still ends up pushing the full data volume to the PFS eventually, which can still lead to long-term congestion if the model is massive and the frequency is high.

F. PAIO (*Macedo et al., FAST '22*)

- **How?**

Data plane framework for building storage middleware with policy-driven I/O enforcement. Provides interception and classification primitives that allow transparent redirection of I/O requests without modifying the application.

- **Relevance**

Provides the interception and classification primitives used in our client stage.

G. PADLL (*Macedo et al., CCGrid '23*)

- **How?**

QoS middleware for HPC storage built on top of PAIO. Demonstrates rate limiting and noisy neighbour mitigation in Lustre environments through a centralized control plane that coordinates I/O across all nodes.

- **Relevance**

Directly informs the design of our orchestrator and its pluggable rate-limiting policies.